



Instituto Tecnológico Superior de Lerdo

Ingeniería Informática

Programación Orientada a Objetos
Actividad de evaluación (ABPj) – Corte 3

ABPj: Sistema de Usuarios con Herencia en Python (un cambio de lenguaje)

Propósito de la evaluación

Investigando un poco el lenguaje, el estudiante desarrollará un pequeño sistema orientado a objetos en **Python** utilizando herencia para modelar distintos tipos de usuarios de un sistema.

Además de programar y ejecutar correctamente el sistema, deberá explicar visualmente toda la solución mediante una **infografía** técnica.

Objetivo

Aplicar el concepto de **herencia** y polimorfismo creando una clase base Usuario y clases derivadas (Admin, Cliente, Invitado), reutilizando atributos y métodos.

Problema

Una plataforma digital necesita controlar distintos tipos de usuarios dentro de su sistema:

- Administradores
- Clientes
- Invitados

Todos comparten información básica, pero cada tipo de usuario tiene comportamientos y permisos distintos.

Tu tarea será desarrollar el sistema utilizando **herencia en Python**.

Requerimientos

1. Crear una clase base llamada Usuario

Atributos:

- nombre
- email

Métodos:

- `mostrar_datos()`
- `acceso_sistema()`

2. Crear las siguientes clases derivadas

Admin

- hereda de `Usuario`
- agregar a la clase:
 - `nivel_acceso`
- y sobrescribir:
 - `acceso_sistema()`

Cliente

- hereda de `Usuario`
- agregar:
 - `puntos`
- sobrescribir:
 - `acceso_sistema()`

Invitado

- hereda de `Usuario`
- sobrescribir:
 - `acceso_sistema()`

3. Desafíos

Agregar:

- método `saludar()`
- validación de email
- menú interactivo
- lista de usuarios
- polimorfismo recorriendo usuarios en una lista

4. Aplicación principal

El programa principal deberá crear al menos:

- 1 administrador
- 1 cliente

- 1 invitado

Deberá mostrar datos de cada usuario y acceso correspondiente de cada uno.

Recuerda hacer:

- Uso correcto de clases
- Uso de herencia
- Uso de constructores (`__init__`)
- Uso de `super()`
- Métodos sobrescritos
- Código ejecutable sin errores
- Comentarios básicos en el código

5. Infografía técnica (obligatoria)

Además del programa funcionando, deberás realizar una **infografía digital** explicando todo el ejercicio y esta debe incluir:

1. Portada (datos generales)
2. Explicación del problema (lo que resuelve el sistema)
3. Diagrama de clases-herencia (UML)
4. Explicación de cada clase (atributos, métodos y propósito)
5. Capturas del código (solo fragmentos importantes)
6. Captura de ejecución (funcionando correctamente)
7. Explicación de herencia, sobreescritura de métodos, uso de `super()`.
8. Conclusiones (¿Qué aprendiste? ¿Qué fue lo más difícil? ¿Dónde podría utilizarse esto en la vida real?)

Entregables

1. Carpeta del proyecto
 - a. `usuario.py`
 - b. `admin.py`
 - c. `cliente.py`
 - d. `invitado.py`
 - e. `main.py`
2. Evidencia de ejecución (a detalle)
3. Infografía
4. Respuesta a las preguntas:
 - a. ¿Qué ventajas ofrece la herencia en comparación con repetir código?
 - b. ¿Qué métodos fueron heredados?
 - c. ¿Por qué sobrescribir `acceso_sistema()`?
 - d. ¿Qué diferencia existe entre un atributo heredado y uno propio?

e. ¿Qué pasaría si todas las clases estuvieran separadas y no heredaran?

Rúbrica de evaluación

Criterio	Excelente 3	Bueno 2	Suficiente 1	Insuficiente 0
Uso de herencia	Correcto y completo	Pequeños errores	Parcial	Incorrecto
Implementación de clases	Bien estructuradas	Algunos detalles	Básicas	Incompletas
Sobrescritura de métodos	Correcta	Parcial	Limitada	Incorrecta
Ejecución del programa	Funciona completamente	Errores menores	Funciona parcialmente	No funciona
Uso de super()	Correcto	Parcial	Confuso	No utilizado
Organización del código	Muy clara	Clara	Regular	Desordenada
Infografía técnica	Completa y clara	Buena	Básica	Incompleta
Explicación oral/demostración	Clara y técnica	Clara	Limitada	Confusa